

Lecture 21-23 Module 5.4 Sequential neural networks

COMP 551 Applied machine learning

Yue Li

Assistant Professor

School of Computer Science

McGill University

March 25, 27 & April 1 & 3, 2025

Outline

Objectives

Sequential data modeling tasks

Recurrent neural network

Autoregressive neural network

Seq2Vec

Seq2Seq

Aligned case

Unaligned case

Backprop through time (or sequence)

LSTM

Attention mechanism

Transformers

Self-attention

Multi-headed attention

Positional encoding

Transformers Encoder

Comparing ConvNets, RNNs, Self-attention

Pre-training deep language models

GPT

BERT

Beyond NLP

Computer Vision

Genomics

Summary

Acknowledgement

- The slides are primarily prepared based on Chapter 15 from the Murphy 2022 textbook
- BERT materials were based on Jay Alammar's post [Illustrated BERT](#)

Learning objectives

- Recurrent neural network prediction tasks (Seq2Vec, Seq2Seq)
- Vanishing gradient problem for modelling long sequence
- Common recurrent units (i.e., LSTM)
- Attention mechanism
- Transformer encoding and decoding
- Pretraining deep language models (GPT and BERT)
- Task-specific fine-tuning for NLP applications

Outline

Objectives

Sequential data modeling tasks

Recurrent neural network

Autoregressive neural network

Seq2Vec

Seq2Seq

Aligned case

Unaligned case

Backprop through time (or sequence)

LSTM

Attention mechanism

Transformers

Self-attention

Multi-headed attention

Positional encoding

Transformers Encoder

Comparing ConvNets, RNNs,
Self-attention

Pre-training deep language models

GPT

BERT

Beyond NLP

Computer Vision

Genomics

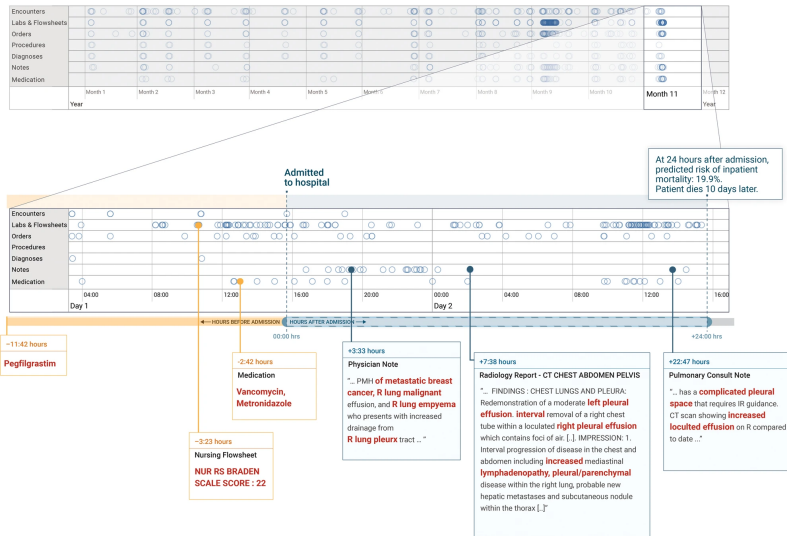
Summary

Sequential data modeling applications

Prediction tasks

- Time series prediction (e.g., **patient outcome prediction**; Red highlighted tokens are the *attended* words to the class label (i.e., risk of patient mortality upon 24 hours of ICU admission).
- Text classification (e.g., sentiment prediction)
- Speech recognition

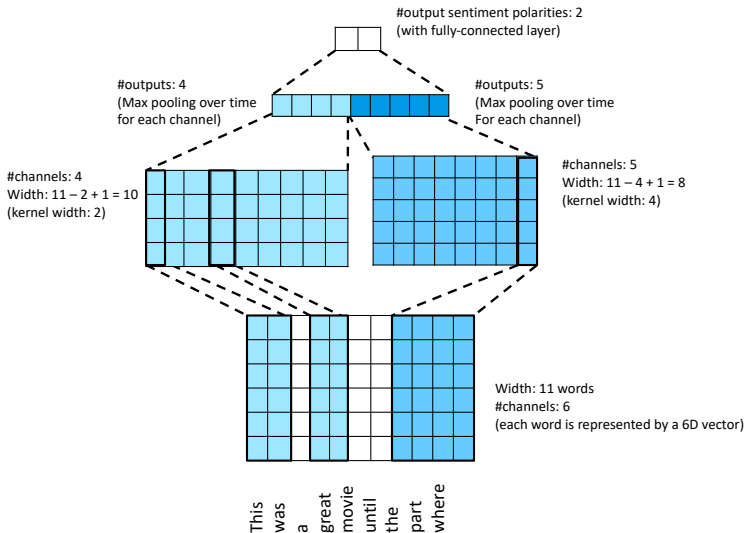
Patient Timeline



Generative tasks

- Machine translation
- Image to text generation
- Text to text (e.g., Q&A)

1D ConvNet for text classification



- In ConvNet, we slide the filter across sequence of text with each word one-hot encoded
- This works for text classification task.
- ConvNet does not “remember” the past context outside of the filter size at each hidden layer (although increasing the receptive field via DNN or dilated convolution helps)
- ConvNet cannot be used to *generate* future sequence

Outline

Objectives

Sequential data modeling tasks

Recurrent neural network

Autoregressive neural network

Seq2Vec

Seq2Seq

Aligned case

Unaligned case

Backprop through time (or sequence)

LSTM

Attention mechanism

Transformers

Self-attention

Multi-headed attention

Positional encoding

Transformers Encoder

Comparing ConvNets, RNNs,
Self-attention

Pre-training deep language models

GPT

BERT

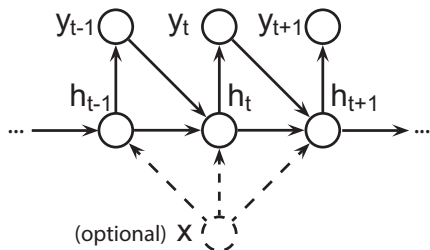
Beyond NLP

Computer Vision

Genomics

Summary

Recurrent neural networks (RNN) is an autoregressive neural network



Training via teacher forcing:

$$\mathbf{h}_t = \varphi([\mathbf{x}; \mathbf{y}_{t-1}] \mathbf{W}_{xh} + \mathbf{h}_{t-1} \mathbf{W}_{hh} + \mathbf{b}_h)$$

$$\hat{\mathbf{y}}_t = \exp(\mathbf{h}_t \mathbf{W}_{hy} + \mathbf{b}) / \sum_c \exp(\mathbf{h}_t \mathbf{w}_{\cdot,c} + b_c)$$

$$\mathcal{L} = - \sum_t \sum_c y_{t,c} \log \hat{y}_{t,c}$$

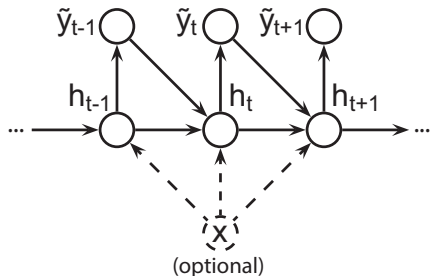
Learnable network weights by backprop:

- $\mathbf{W}_{xh}$: input-to-hidden
- $\mathbf{W}_{hh}$: hidden-to-hidden
- $\mathbf{W}_{hy}$: hidden-to-output
- $\varphi(h)$: activation function e.g., ReLu.

Inference (generating text):

1. Predict $\hat{\mathbf{y}}_t = f(\mathbf{h}_t; \mathbf{W}_{hy})$
2. Sample $\tilde{y}_t \sim \text{Cat}(\hat{\mathbf{y}}_t) = \prod_c \hat{y}_{t,c}^{[\tilde{y}_t=c]}$
3. Feed $\tilde{\mathbf{y}}_t$ (one-hot) into hidden state to get $\mathbf{h}_{t+1} = f(\mathbf{h}_t, \tilde{\mathbf{y}}_t, \mathbf{x}; \mathbf{W}_{xh}, \mathbf{W}_{hh})$
4. Repeat 1-3 until generating S characters

Using RNN to generate text

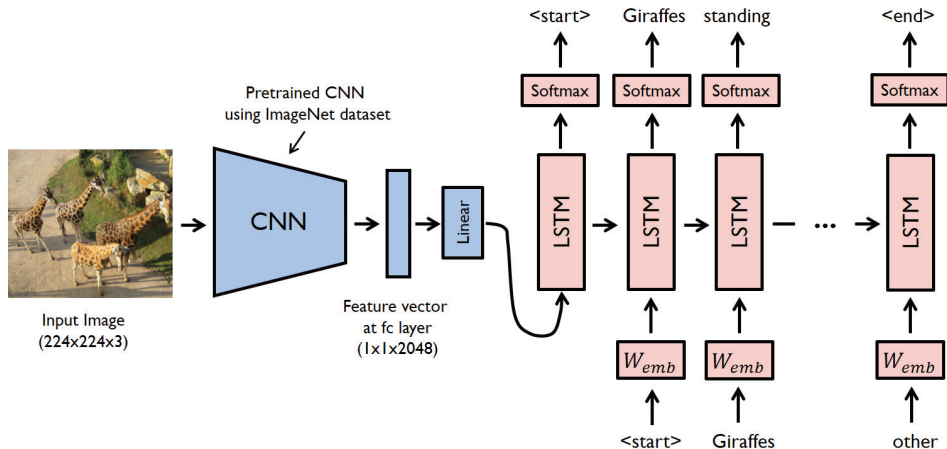


500 characters generated by an RNN, which was trained on *The Time Machine* authored by H. G. Wells (32K words, 170k characters) from only the initial word “the”:

“the githa some thong the time traveller held in his hand was a glittering metallic framework scarcely larger than a small clock and verydelicately made there was ivory in it and the latter than s bettyre tat howhong s ie time thave ler simk you a dimensions le ghat dionthat shall travel indifferently in any direction of space and timeas the driver determinesfilby contented himself with laughterbut i have experimental verification said the time travellerit would be remarkably convenient for the histo”

- The output $\mathbf{y}_t$ implicitly depends on all past observations
- RNN can have unbounded memory (i.e., it can generate text forever) and is theoretically as powerful as a Turing machine (i.e., a universal machine with infinite I/O tape)

Image captioning using ConvNet+RNN architecture (Karpathy & Li; CVPR 2015)



W_{emb} is the K -dimensional word embedding for the sampled one-hot token. Based on the ConvNet feature map and the artificially introduced **<start>** token, the RNN will generate sequence of text that describe the input image. We will learn LSTM in this module.

Outline

Objectives

Sequential data modeling tasks

Recurrent neural network

Autoregressive neural network

Seq2Vec

Seq2Seq

Aligned case

Unaligned case

Backprop through time (or sequence)

LSTM

Attention mechanism

Transformers

Self-attention

Multi-headed attention

Positional encoding

Transformers Encoder

Comparing ConvNets, RNNs,

Self-attention

Pre-training deep language models

GPT

BERT

Beyond NLP

Computer Vision

Genomics

Summary

Seq2Vec (sequence classification)

Goal: predict output vector $\mathbf{y}$ given a sequence of input $\mathbf{x}_{1:T}$ (e.g., diagnosing patients after observing T time points of their hospital stays).

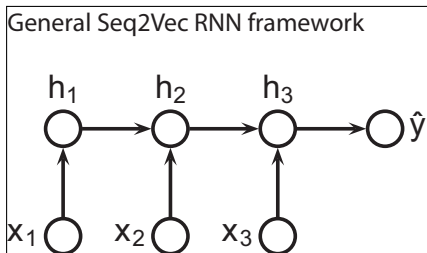
1. At each time point t , $\mathbf{x}_t$ is an observed D -dimensional feature vector.
2. At time point 1, compute the initial hidden state $\mathbf{h}_1$:
4. Finally, make prediction based on the hidden state in the last time point:

$$\hat{\mathbf{y}} = \sigma(\mathbf{h}_T \mathbf{W}_{hy} + \mathbf{b}_y)$$

$$\mathbf{h}_1 = \varphi(\mathbf{x}_1 \mathbf{W}_{xh} + \mathbf{b}_h)$$

3. At time point $t > 1$, based on the input $\mathbf{x}_t$ and the *hidden state* $\mathbf{h}_{t-1}$ at time $t - 1$, compute the hidden state:

$$\mathbf{h}_t = \varphi(\mathbf{x}_t \mathbf{W}_{xh} + \mathbf{h}_{t-1} \mathbf{W}_{hh} + \mathbf{b}_h)$$



Bi-directional RNN for Seq2Vec task

We can extend the above RNN by making the hidden state depend on both the past and future context:

1. Hidden states at layer 1 goes from past to the future
2. Hidden states at layer 2 goes from future to the past
3. Hidden states at layer 3 concatenates the hidden states at layer 1 and 2
4. All time points are average-pooled into one vector as for prediction

Translating the above into Greek:

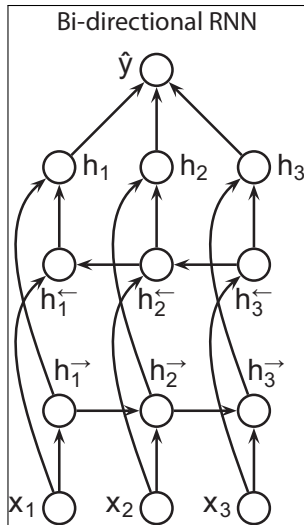
$$\mathbf{h}_t^{\rightarrow} = \varphi(\mathbf{x}_t \mathbf{W}_{xh}^{\rightarrow} + \mathbf{h}_{t-1}^{\rightarrow} \mathbf{W}_{hh}^{\rightarrow} + \mathbf{b}_h^{\rightarrow}) \quad (\text{past} \rightarrow \text{future})$$

$$\mathbf{h}_t^{\leftarrow} = \varphi(\mathbf{x}_t \mathbf{W}_{xh}^{\leftarrow} + \mathbf{h}_{t+1}^{\leftarrow} \mathbf{W}_{hh}^{\leftarrow} + \mathbf{b}_h^{\leftarrow}) \quad (\text{past} \leftarrow \text{future})$$

$$\mathbf{h}_t = [\mathbf{h}_t^{\rightarrow}, \mathbf{h}_t^{\leftarrow}] \quad (\text{concat})$$

$$\bar{\mathbf{h}} = \frac{1}{T} \sum_{t=1}^T \mathbf{h}_t \quad (\text{average pooling})$$

$$\hat{\mathbf{y}} = \sigma(\bar{\mathbf{h}} \mathbf{W}) \quad (\text{multi-label prediction})$$



Outline

Objectives

Sequential data modeling tasks

Recurrent neural network

Autoregressive neural network

Seq2Vec

Seq2Seq

Aligned case

Unaligned case

Backprop through time (or sequence)

LSTM

Attention mechanism

Transformers

Self-attention

Multi-headed attention

Positional encoding

Transformers Encoder

Comparing ConvNets, RNNs,

Self-attention

Pre-training deep language models

GPT

BERT

Beyond NLP

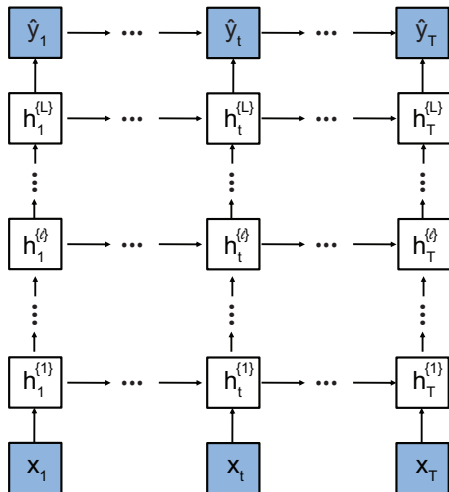
Computer Vision

Genomics

Summary

Seq2Seq (sequence translation) - aligned input-output sequences

Deep seq2seq L-layer RNN with aligned input-to-output sequence:



$$\hat{y}_t = \sigma(\mathbf{h}_t^{\{L\}} \mathbf{W}_{hy}^{\{o\}} + \hat{y}_{t-1} \mathbf{W}_{yy}^{\{o\}} + \mathbf{b}_y^{\{o\}})$$

$$\hat{y}_1 = \sigma(\mathbf{h}_1^{\{L\}} \mathbf{W}_{hy}^{\{o\}} + \mathbf{b}_y^{\{o\}})$$

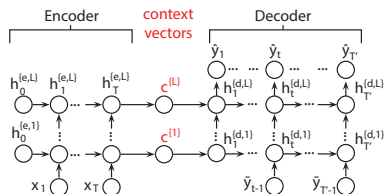
$$\mathbf{h}_t^{\{\ell\}} = \varphi(\mathbf{h}_t^{\{\ell-1\}} \mathbf{W}_{xh}^{\{\ell\}} + \mathbf{h}_{t-1}^{\{\ell\}} \mathbf{W}_{hh}^{\{\ell\}} + \mathbf{b}_h^{\{\ell\}})$$

$$\mathbf{h}_1^{\{\ell\}} = \varphi(\mathbf{h}_1^{\{\ell-1\}} \mathbf{W}_{xh}^{\{\ell\}} + \mathbf{b}_h^{\{\ell\}})$$

$$\mathbf{h}_t^{\{1\}} = \varphi(\mathbf{x}_t \mathbf{W}_{xh}^{\{1\}} + \mathbf{h}_{t-1}^{\{1\}} \mathbf{W}_{hh}^{\{1\}} + \mathbf{b}_h^{\{1\}})$$

$$\mathbf{h}_1^{\{1\}} = \varphi(\mathbf{x}_1 \mathbf{W}_{xh}^{\{1\}} + \mathbf{b}_h^{\{1\}})$$

Seq2Seq: unaligned IO sequences for Neural Machine Translation (NMT)



- Map from a sequence of length T to another of length $T' \neq T$.
- Encode the input sequence into a *context vector* $\mathbf{c}$ at each layer:

$$\mathbf{h}_t^{\{e,\ell\}} = \varphi(\mathbf{h}_t^{\{e,\ell-1\}} \mathbf{W}_{xh}^{\{e,\ell\}} + \mathbf{h}_{t-1}^{\{e,\ell\}} \mathbf{W}_{hh}^{\{e,\ell\}} + \mathbf{b}^{\{e,\ell\}})$$

$$\mathbf{c}^{\{\ell\}} = \varphi(\mathbf{h}_T^{\{e,\ell\}} \mathbf{W}_{hc}^{\{e,\ell\}} + \mathbf{b}_c^{\{e\}})$$

- Decode the first target word:

$$\mathbf{h}_1^{\{d,\ell\}} = \varphi(\mathbf{c}^{\{\ell\}} \mathbf{W}_{ch}^{\{d,\ell\}} + \mathbf{h}_1^{\{d,\ell-1\}} \mathbf{W}_{hh}^{\{d,\ell\}} + \mathbf{b}_h^{\{d,\ell\}})$$

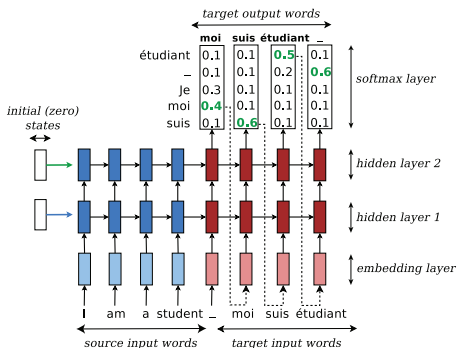
$$\hat{\mathbf{y}}_1 = \varphi(\mathbf{h}_1^{\{d,L\}} \mathbf{W}_{hy}^{\{d,L\}} + \mathbf{b}_y^{\{d\}})$$

- For decoding word at position t , sample a word $\tilde{\mathbf{y}}_{t-1}$ (one-hot) from the word probabilities $\hat{\mathbf{y}}_{t-1}$ at $t-1$ and then feed it into the RNN-decoder along with the previous hidden state:

$$\mathbf{h}_t^{\{d,1\}} = \varphi(\tilde{\mathbf{y}}_{t-1} \mathbf{W}_{yh}^{\{d\}} + \mathbf{h}_{t-1}^{\{d,1\}} \mathbf{W}_{hh}^{\{d\}} + \mathbf{b}_h^{\{d,1\}})$$

$$\mathbf{h}_t^{\{d,\ell\}} = \varphi(\mathbf{h}_t^{\{d,\ell-1\}} \mathbf{W}_{xh}^{\{d,\ell\}} + \mathbf{h}_{t-1}^{\{d,\ell\}} \mathbf{W}_{hh}^{\{d,\ell\}} + \mathbf{b}_h^{\{d,\ell\}})$$

$$\hat{\mathbf{y}}_t = \varphi(\mathbf{h}_t^{\{d,L\}} \mathbf{W}_{hy}^{\{o\}} + \mathbf{b}_y^{\{d\}})$$



Outline

Objectives

Sequential data modeling tasks

Recurrent neural network

Autoregressive neural network

Seq2Vec

Seq2Seq

Aligned case

Unaligned case

Backprop through time (or sequence)

LSTM

Attention mechanism

Transformers

Self-attention

Multi-headed attention

Positional encoding

Transformers Encoder

Comparing ConvNets, RNNs,
Self-attention

Pre-training deep language models

GPT

BERT

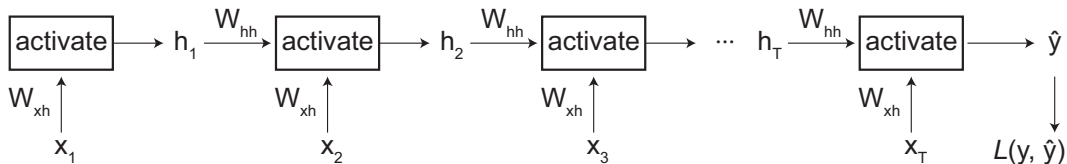
Beyond NLP

Computer Vision

Genomics

Summary

Backprop through time (or sequence)



Forward pass:

For $t = 1, \dots, T$, $\mathbf{h}_t = \sigma(\mathbf{x}_t \mathbf{W}_{xh} + \mathbf{h}_{t-1} \mathbf{W}_{hh} + \mathbf{b}_h) \Rightarrow \hat{y} = \sigma(\mathbf{h}_T \mathbf{W}_{ho} + \mathbf{b}_y) \Rightarrow \mathcal{L}(y, \hat{y})$

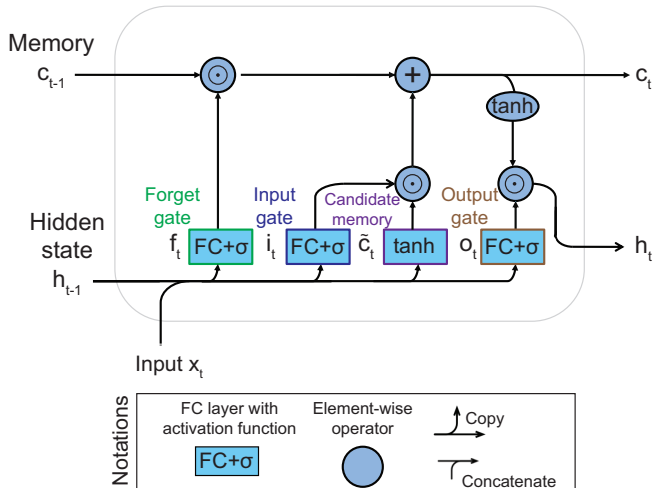
Backprop (to simplify notation, consider $\mathbf{w}_h$ is a stacked, flattened version of $\mathbf{W}_{hh}$ and $\mathbf{W}_{hx}$):

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial \mathbf{w}_h} &= \frac{\partial \mathcal{L}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial \mathbf{h}_T} \frac{\partial \mathbf{h}_T}{\partial \mathbf{a}_T} \frac{\partial \mathbf{a}_T}{\partial \mathbf{h}_{T-1}} \dots \frac{\partial \mathbf{h}_2}{\partial \mathbf{a}_2} \frac{\partial \mathbf{a}_2}{\partial \mathbf{h}_1} \frac{\partial \mathbf{h}_1}{\partial \mathbf{a}_1} \frac{\partial \mathbf{a}_1}{\partial \mathbf{w}_h} \\ &= \frac{\partial \mathcal{L}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial \mathbf{h}_T} \prod_{t=1}^{T-1} \underbrace{\frac{\partial \mathbf{h}_{T-t+1}}{\partial \mathbf{a}_{T-t+1}} \frac{\partial \mathbf{a}_{T-t+1}}{\partial \mathbf{h}_{T-t}}}_{\frac{\partial \mathbf{h}_{T-t+1}}{\partial \mathbf{h}_{T-t}}} \frac{\partial \mathbf{h}_1}{\partial \mathbf{a}_1} \frac{\partial \mathbf{a}_1}{\partial \mathbf{w}_h} \end{aligned}$$

- Multiplying long sequence of $\frac{\partial \mathbf{h}_t}{\partial \mathbf{a}_t}$'s causes vanishing gradient problem
- We face similar problem with deep layers (i.e., large L) and tackle it with ReLU activation function, residual/skip connections, layer normalization, etc

More effectively, we can design specialized hidden layer to tackle this problem in RNN.

Long short term memory (LSTM) unit



Forget gate:

$$\mathbf{f}_t = \sigma(\mathbf{x}_t \mathbf{W}_{xf} + \mathbf{h}_{t-1} \mathbf{W}_{hf} + b_f)$$

Input gate:

$$\mathbf{i}_t = \sigma(\mathbf{x}_t \mathbf{W}_{xi} + \mathbf{h}_{t-1} \mathbf{W}_{hi} + b_i)$$

Candidate memory:

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{x}_t \mathbf{W}_{xc} + \mathbf{h}_{t-1} \mathbf{W}_{hc} + b_c)$$

Output gate:

$$\mathbf{o}_t = \sigma(\mathbf{x}_t \mathbf{W}_{xo} + \mathbf{h}_{t-1} \mathbf{W}_{ho} + b_o)$$

Update cell:

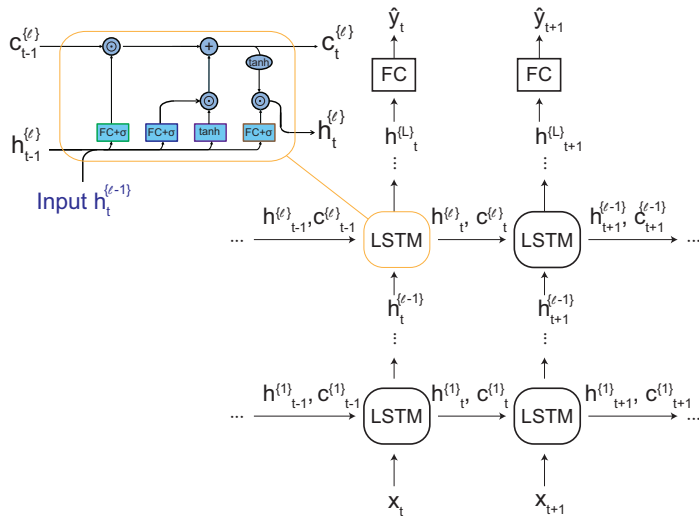
$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t$$

Final hidden state at t :

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$$

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} = \frac{\partial}{\partial \mathbf{h}_{t-1}} \mathbf{o}_t \odot \tanh(\mathbf{c}_t) = \frac{\partial \mathbf{o}_t}{\partial \mathbf{h}_{t-1}} \odot \tanh(\mathbf{c}_t) + \mathbf{o}_t \odot \frac{\partial \tanh(\mathbf{c}_t)}{\partial \mathbf{h}_{t-1}}$$

Going deep with L LSTM layers



Forget gate:

$$\mathbf{f}_t^{\{\ell\}} = \sigma(\mathbf{h}_t^{\{\ell-1\}} \mathbf{W}_{xf} + \mathbf{h}_{t-1}^{\{\ell\}} \mathbf{W}_{hf} + b_f)$$

Input gate:

$$\mathbf{i}_t^{\{\ell\}} = \sigma(\mathbf{h}_t^{\{\ell-1\}} \mathbf{W}_{xi} + \mathbf{h}_{t-1}^{\{\ell\}} \mathbf{W}_{hi} + b_i)$$

Candidate memory:

$$\tilde{\mathbf{c}}_t^{\{\ell\}} = \tanh(\mathbf{h}_t^{\{\ell-1\}} \mathbf{W}_{xc} + \mathbf{h}_{t-1}^{\{\ell\}} \mathbf{W}_{hc} + b_c)$$

Output gate:

$$\mathbf{o}_t^{\{\ell\}} = \sigma(\mathbf{h}_t^{\{\ell-1\}} \mathbf{W}_{xo} + \mathbf{h}_{t-1}^{\{\ell\}} \mathbf{W}_{ho} + b_o)$$

Update cell:

$$\mathbf{c}_t^{\{\ell\}} = \mathbf{f}_t^{\{\ell\}} \odot \mathbf{c}_{t-1}^{\{\ell\}} + \mathbf{i}_t^{\{\ell\}} \odot \tilde{\mathbf{c}}_t^{\{\ell\}}$$

Final hidden state at t :

$$\mathbf{h}_t^{\{\ell\}} = \mathbf{o}_t^{\{\ell\}} \odot \tanh(\mathbf{c}_t^{\{\ell\}})$$

Outline

Objectives

Sequential data modeling tasks

Recurrent neural network

Autoregressive neural network

Seq2Vec

Seq2Seq

Aligned case

Unaligned case

Backprop through time (or sequence)

LSTM

Attention mechanism

Transformers

Self-attention

Multi-headed attention

Positional encoding

Transformers Encoder

Comparing ConvNets, RNNs,
Self-attention

Pre-training deep language models

GPT

BERT

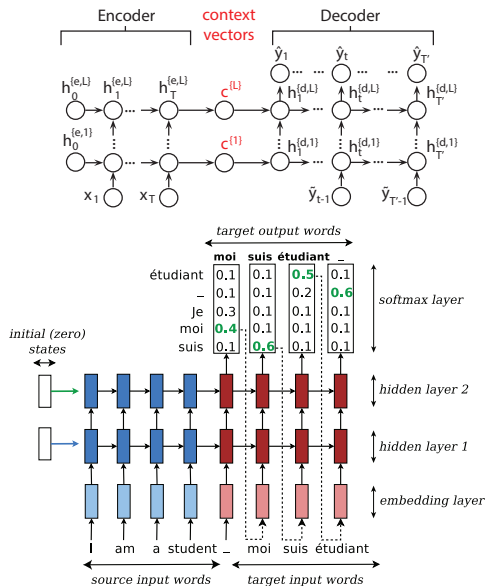
Beyond NLP

Computer Vision

Genomics

Summary

How to make the decoder pay *attention* to the input words



- The above seq2seq RNN model feeds to the decoder a fixed context vector $\mathbf{c}$ and the decoder does not have direct access to the input words.
- This may generate poor translation because the word being translated needs to “pay attention” to the input words from the source language.
- But which inputs should the decoder attend? We will need to infer the *alignment* between the source sequence and target sequence.

Attention mechanism generates *dynamic* context for each target word

1. We first compute the unnormalized attention weight between the decoder hidden state at $t - 1$ and the encoder hidden state for every input word $i \in \{1, \dots, T\}$:

$$\tilde{a}_{t-1,i} = \sum_{k=1}^D h_{t-1,k}^{\{d\}} h_{i,k}^{\{e\}} = \mathbf{h}_{t-1}^{\{d\}} (\mathbf{h}_i^{\{e\}})^\top$$

2. Apply softmax to normalize the attention weight over all of the T input words:

$$a_{t-1,i} = \frac{\exp(\tilde{a}_{t-1,i})}{\sum_{i=1}^T \exp(\tilde{a}_{t-1,i})}$$

3. Compute the *dynamic context vector*:

$$\mathbf{c}_t = \sum_{i=1}^T a_{t-1,i} \mathbf{h}_i^{\{e\}}$$

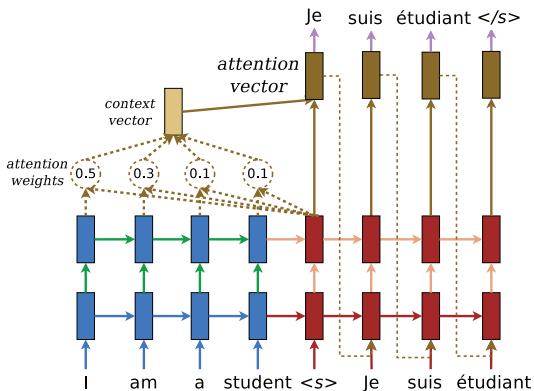
4. Generate hidden state at t using concatenated context $\mathbf{c}_t$, previously generated target word $\tilde{\mathbf{y}}_{t-1}$, and previous hidden state $\mathbf{h}_{t-1}^{\{d\}}$:

$$\mathbf{h}_t^{\{d\}} = \phi([\mathbf{c}_t, \tilde{\mathbf{y}}_{t-1}, \mathbf{h}_{t-1}^{\{d\}}] \mathbf{W}^{\{d\}})$$

Attention mechanism generates dynamic context for each target word

Let's further examine the attention formulation:

Seq2Seq NMT with attention



$$a_{t-1,i}^{d,\ell} = \frac{\exp(\mathbf{h}_{t-1}^{d,\ell} (\mathbf{h}_i^{e,\ell})^\top)}{\sum_{i=1}^T \exp(\mathbf{h}_{t-1}^{d,\ell} (\mathbf{h}_i^{e,\ell})^\top)}$$

$$\mathbf{c}_t^{d,\ell} = \sum_{i=1}^T a_{t-1,i}^{d,\ell} \mathbf{h}_i^{e,\ell}$$

$$\mathbf{h}_t^{d,\ell} = \phi([\mathbf{c}_t^{d,\ell}, \mathbf{h}_t^{d,\ell-1}, \mathbf{h}_{t-1}^{d,\ell}] \mathbf{W}^{d,\ell})$$

- The decoder's word embedding $\mathbf{h}_{t-1}^{d,\ell}$ is referred to as "query" $\mathbf{q} \in \mathbb{R}^D$ or in matrix form $\mathbf{Q} \in \mathbb{R}^{M \times D}$
- The encoder's word embedding $\mathbf{h}_i^{e,\ell}$ is referred to as "key": $\mathbf{k} \in \mathbb{R}^D$ or in matrix form $\mathbf{K} \in \mathbb{R}^{T \times D}$
- The encoder's word embedding $\mathbf{h}_i^{e,\ell}$ is referred to as "value" $\mathbf{v} \in \mathbb{R}^D$ or in matrix form $\mathbf{V} \in \mathbb{R}^{T \times D}$

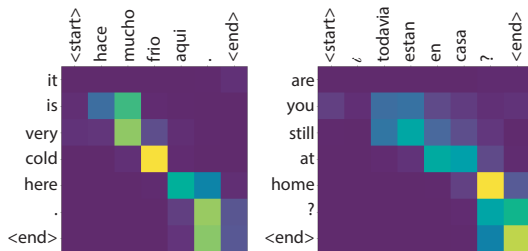
Attention mechanism generates dynamic context for each target word

To translate T words into M words, we can therefore compute the context matrix as:

$$\mathbf{C} = \text{Attn}(\mathbf{Q}, \mathbf{K})\mathbf{V} = \mathcal{S} \left(\frac{\mathbf{QK}^\top}{\sqrt{D}} \right) \mathbf{V} \in \mathbb{R}^{M \times D}$$

where $\mathcal{S}$ is softmax function over the T input words (i.e., softmax each row). We can visualize the $M \times T$ attention matrix $\mathbf{A} = \mathcal{S} \left(\frac{\mathbf{QK}^\top}{\sqrt{D}} \right)$ in a heatmap as below.

Attention heatmaps generated while translating Spanish to English



Interestingly, when generating the output token “home”, the model should attend to the input token “casa”, but in fact it seems to attend to the input token “?”.

Outline

Objectives

Sequential data modeling tasks

Recurrent neural network

Autoregressive neural network

Seq2Vec

Seq2Seq

Aligned case

Unaligned case

Backprop through time (or sequence)

LSTM

Attention mechanism

Transformers

Self-attention

Multi-headed attention

Positional encoding

Transformers Encoder

Comparing ConvNets, RNNs,
Self-attention

Pre-training deep language models

GPT

BERT

Beyond NLP

Computer Vision

Genomics

Summary

Self-attention learns attention between words in the input sentence

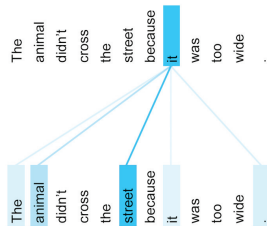
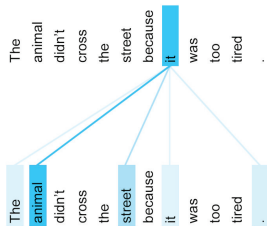
- Given a $N \times V$ one-hot encoded matrix $\mathbf{X}$ for a sentence of length N , we compute the embedding of the *query*, *key*, and *value*. The former two form the attention, which multiplies the latter to form the word embedding:

$$\text{Query: } \mathbf{Q} = \mathbf{XW}^{(q)}; \quad \text{Key: } \mathbf{K} = \mathbf{XW}^{(k)}; \quad \text{Value: } \mathbf{V} = \mathbf{XW}^{(v)}$$

$$\text{Self-attention: } \mathbf{A} = \text{Attn}(\mathbf{Q}, \mathbf{K}); \quad \tilde{a}_{i,j} = \mathbf{q}_i \mathbf{k}_j^\top / \sqrt{D}; \quad a_{i,j} = \frac{\exp(\tilde{a}_{i,j})}{\sum_j \exp(\tilde{a}_{i,j})}$$

$$\text{Word embedding: } \mathbf{Z} = \mathbf{AV}$$

- For multi-layer embedding, $\mathbf{Z}^{\{\ell\}}$ is used as input for $\mathbf{Q}$, $\mathbf{K}$, $\mathbf{V}$ in the next layer $\mathbf{Z}^{\{\ell+1\}}$
- The example on the right shows the word “it” attends to different words based on the context.

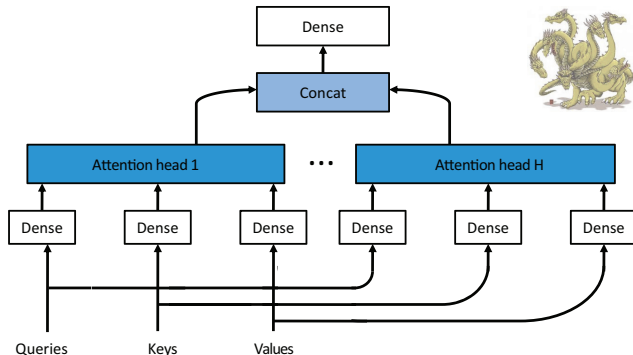


Multi-headed attention (MHA)

Besides going deep, we can also go wide by having MHA with each head indexed by $i \in \{1, \dots, h\}$:

Query: $\mathbf{Q}_i = \mathbf{X}\mathbf{W}_i^{(q)}$; Key: $\mathbf{K}_i = \mathbf{X}\mathbf{W}_i^{(k)}$; Value: $\mathbf{V}_i = \mathbf{X}\mathbf{W}_i^{(v)}$

Self-attention: $\mathbf{A}_i = \text{Attn}(\mathbf{Q}_i, \mathbf{K}_i)$; Word embedding: $\mathbf{Z}_i = \mathbf{A}_i \mathbf{V}_i$



We then concatenate the outputs from all h attention heads and feed them into a dense layer (i.e., a full-connected layer) to get the final embedding:

$$\mathbf{Z} = [\mathbf{Z}_1, \dots, \mathbf{Z}_h] \mathbf{W}_o \in \mathbb{R}^{N \times D}$$

where $\mathbf{W}_o$ is a $(\sum_{i=1}^h D_i) \times D$ matrix.

Positional encoding (PE)

- Attention is permutation invariant and therefore ignores the input word ordering. To make it aware of the word position we can use PE to represent the sentence with $\mathbf{P} \in \mathbb{R}^{N \times D}$.
- A commonly used PE is a set of sinusoidal basis functions. To encode the i^{th} word position using the j^{th} encoding dimension for $j \in \{0, \dots, D/2\}$, we have

$$p_{i,2j} = \sin\left(\frac{i}{C^{2j/D}}\right), \quad p_{i,2j+1} = \cos\left(\frac{i}{C^{2j/D}}\right),$$

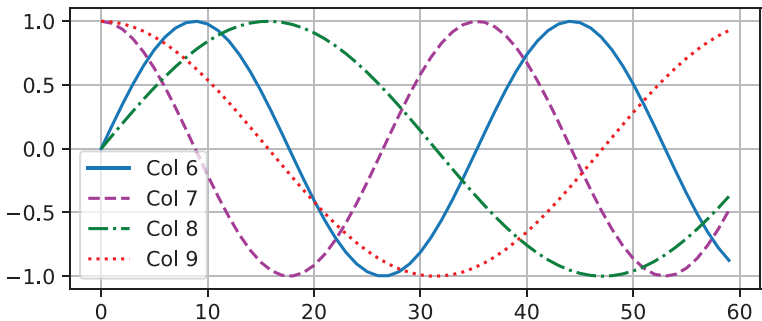
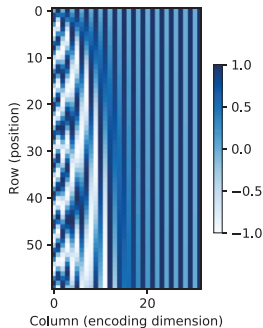
where C (e.g., $C = 10,000$) is some large constant that is not important here.

- For example, if $D = 4$, the encoding for the i position is:

$$\mathbf{p}_i = \left[\sin\left(\frac{i}{C^{0/4}}\right), \cos\left(\frac{i}{C^{0/4}}\right), \sin\left(\frac{i}{C^{2/4}}\right), \cos\left(\frac{i}{C^{2/4}}\right) \right]$$

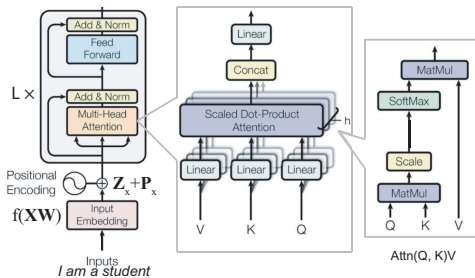
- We can then combine the PE $\mathbf{P} \in \mathbb{R}^{N \times D}$ with the original word embedding $\mathbf{Z} \in \mathbb{R}^{N \times D}$ by either concatenating them $\mathbf{Z}^* = [\mathbf{Z}, \mathbf{P}]$ or adding them $\mathbf{Z}^* = \mathbf{Z} + \mathbf{P}$.

PE matrix for a sequence of length $N = 60$ and $D = 32$



$$\mathbf{p}_i = \left[\sin\left(\frac{i}{C^{0/32}}\right), \cos\left(\frac{i}{C^{0/32}}\right), \dots, \sin\left(\frac{i}{C^{16/32}}\right), \cos\left(\frac{i}{C^{16/32}}\right) \right], \quad \forall i \in \{1, \dots, 60\}$$

Putting it together: Transformer Encoder



$$Q_i = Z^{(\ell)} W_i^{(q)}, \quad K_i = Z^{(\ell)} W_i^{(k)}, \quad V_i = Z^{(\ell)} W_i^{(v)}$$

$$A_i = \text{softmax} \left(\frac{Q_i K_i^T}{\sqrt{D/H}} \right) \left(a_{n,m} = \frac{\exp(q_n k_m^T)}{\sum_{m'} \exp(q_n k_{m'}^T)} \right)$$

$$Z_i = A_i V_i \quad \text{for } i \in \{1, \dots, h\}$$

$$Z^* = [Z_1, \dots, Z_h] W_h$$

$$\tilde{Z} = \text{LayerNorm}(Z^* + Z^{(\ell)})$$

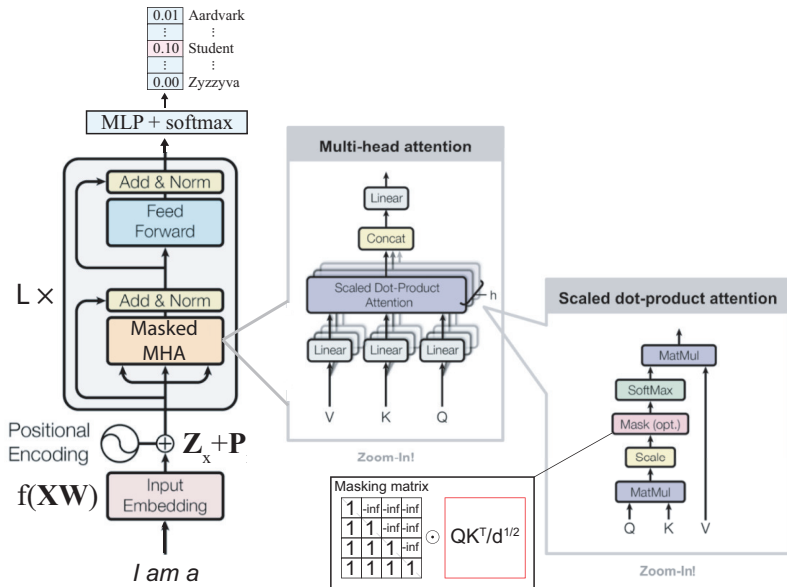
$$Z' = \text{FeedForward}(\tilde{Z}) = \begin{bmatrix} \text{ReLU}(\tilde{Z}_1 W_{FF} + b) \\ \text{ReLU}(\tilde{Z}_2 W_{FF} + b) \\ \vdots \\ \text{ReLU}(\tilde{Z}_N W_{FF} + b) \end{bmatrix}$$

$$Z^{(\ell+1)} = \text{LayerNorm}(Z' + \tilde{Z})$$

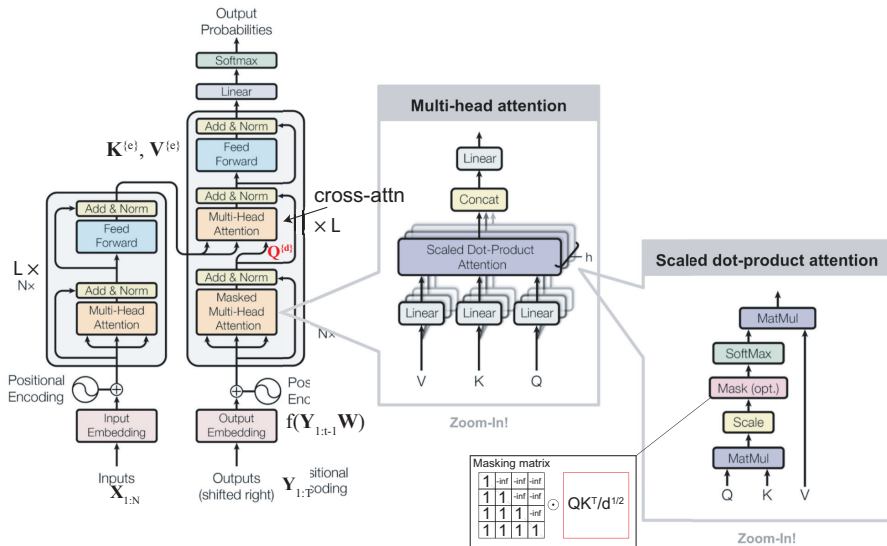
```
def EncoderBlock(X):
    Ze = LayerNorm(MHA(Q=X, K=X, V=X) + X)
    E = LayerNorm(FeedForward(Ze) + Ze)
    return E
```

```
def Encoder(X, L):
    Ze = POS(Embed(X))
    for l in range(L):
        Ze = EncoderBlock(Ze)
    return Ze
```

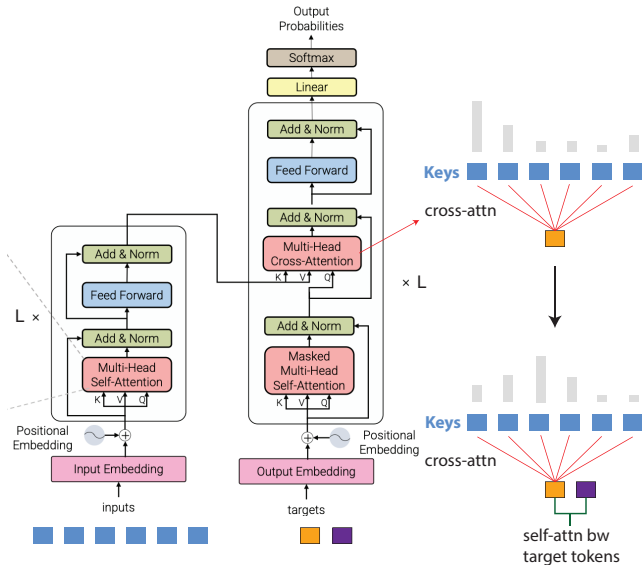
Transformer Decoder of masked self-attention for next token prediction



Original transformer encoder-decoder architecture for language translation



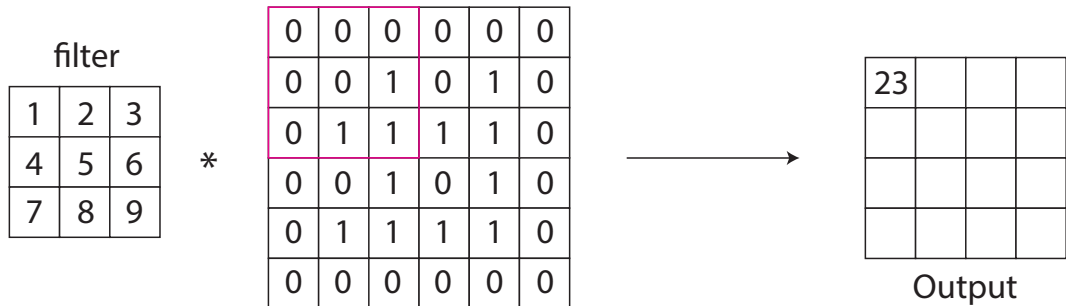
Why is Transformer attention so expressive?



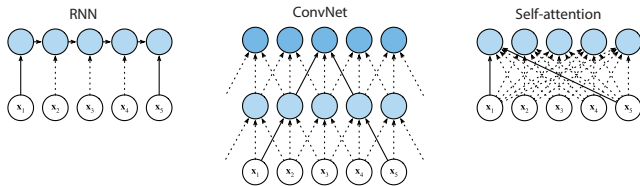
- During *inference*, the transformer attention is data-dependent and dynamic
- Because of the self-attention among the target tokens, their cross-attention to the source tokens change as more words being translated.

In contrast, standard convolution is not data-dependent

Same kernel applied over all of the input values



Comparing ConvNets, RNNs, Self-attention model



- For RNN, to map $\mathbf{X} \in \mathbb{R}^{N \times D}$ to $\mathbf{H} \in \mathbb{R}^{N \times D}$, we take the inner product of $\mathbf{x}_t$ or $\mathbf{h}_t \in \mathbb{R}^{1 \times D}$ and $\mathbf{W} \in \mathbb{R}^{D \times D}$ N times, the time complexity is $O(ND^2)$. It's inherently sequential operation that takes $O(N)$ steps. Information flows from one node to another node also takes $O(N)$.
- For 1D ConvNet with filter size K and D IO channels, the complexity is $O(KND^2)$ and we need a stack of N/K layers or $\log_K N$ if we use dilated convolution to ensure all pairs can communicate.
- For self-attention, we compute $\mathbf{X}\mathbf{X}^\top$, which takes $O(DN^2)$, which is expensive for long sequence.
- ConvNet and self-attn can be done *in parallel* at each layer and therefore no sequential ops.

Layer type	Complexity	Sequential ops.	Max. path length
RNN	$O(ND^2)$	$O(N)$	$O(N)$
ConvNets	$O(KND^2)$	$O(1)$	$O(\log_k N)$
Self-attention	$O(N^2D)$	$O(1)$	$O(1)$

Table: N sequence length; D : feature dimension; K : kernel size for ConvNet.

Outline

Objectives

Sequential data modeling tasks

Recurrent neural network

Autoregressive neural network

Seq2Vec

Seq2Seq

Aligned case

Unaligned case

Backprop through time (or sequence)

LSTM

Attention mechanism

Transformers

Self-attention

Multi-headed attention

Positional encoding

Transformers Encoder

Comparing ConvNets, RNNs,
Self-attention

Pre-training deep language models

GPT

BERT

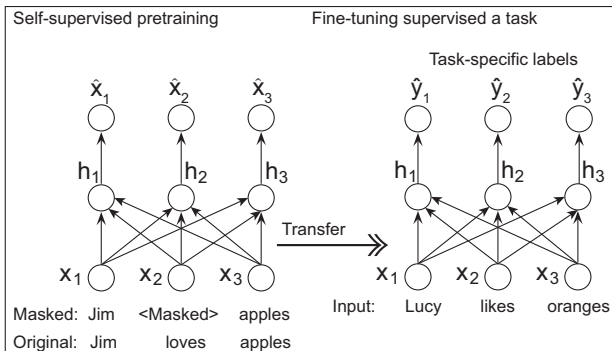
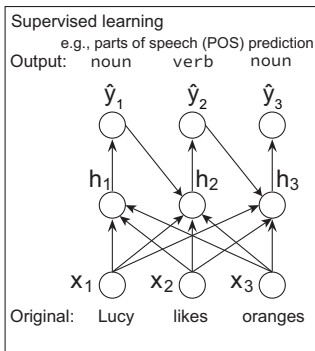
Beyond NLP

Computer Vision

Genomics

Summary

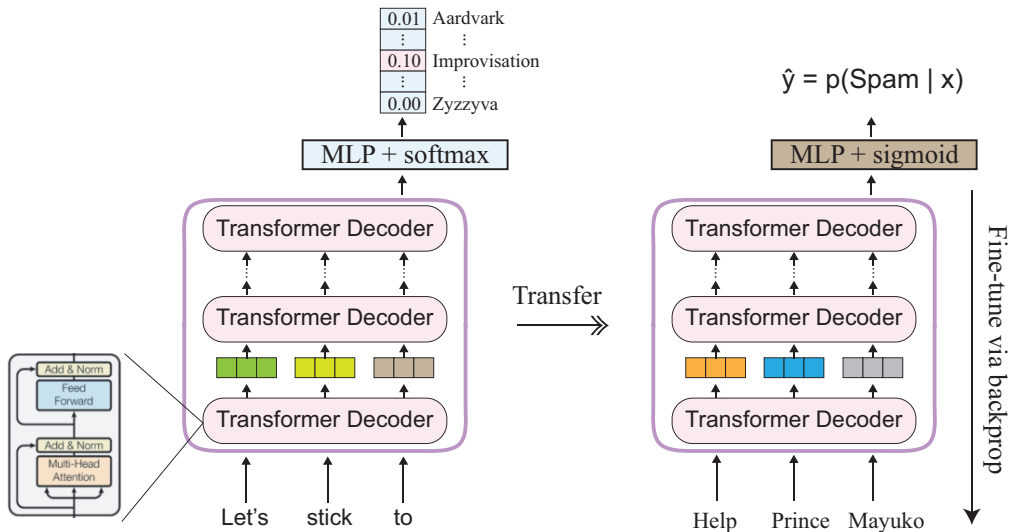
Self-supervised pre-training of a language model



- In supervised learning, we train a model directly to predict target label
- In *self-supervised* learning, we train a model on the input data *without label*. One type of self-supervised learning is to predict randomly masked words.
- The weights for the trained self-supervised model can then be *transferred* to supervised model for prediction. The self-supervised training is called *pretraining* and the subsequent supervised learning is called *task-specific fine-tuning*.

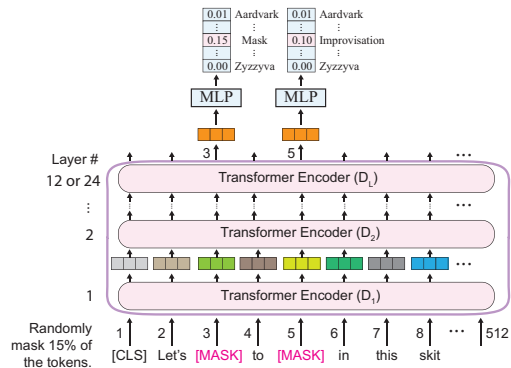
OpenAI: Generative Pre-training Transformer (GPT)

GPT is trained to predict next word using a stack of 12 Transformer *Decoder* blocks

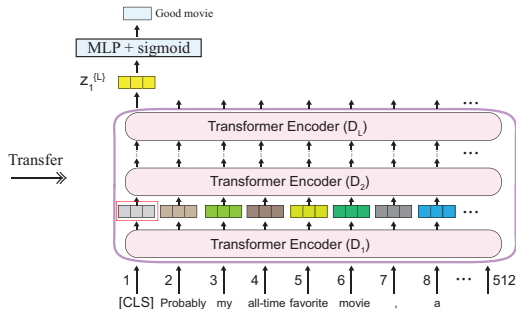


BERT: Bidirectional Encoder Representations from Transformers [Dev+19]

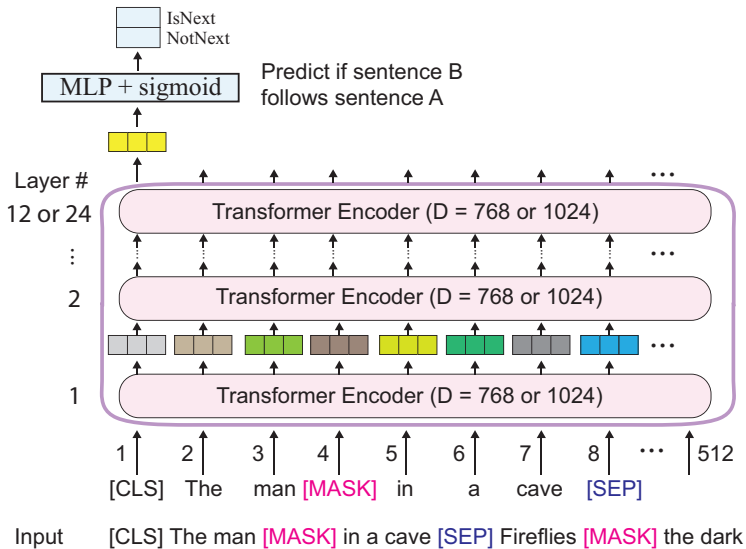
Pre-training via masked language model (MLM):



Task-specific fine-tuning:

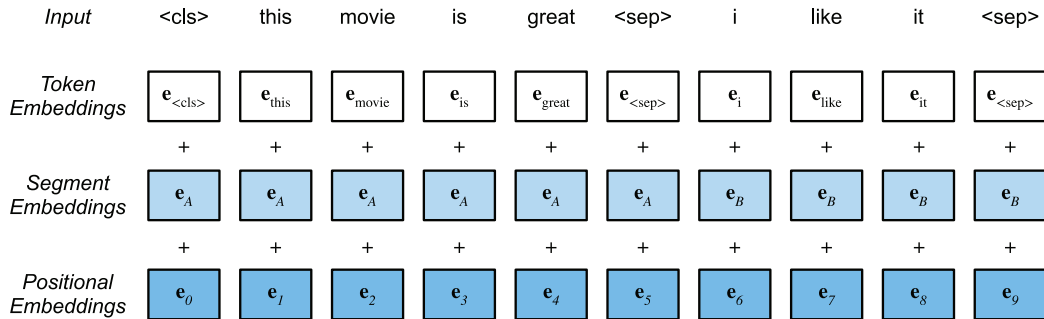


Pre-training task 2: next sentence prediction (NSP)



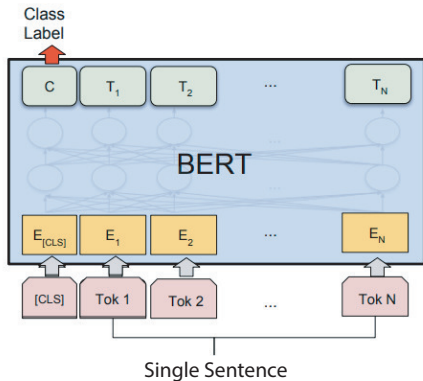
Pre-training NSP details

- The model takes as input: CLS A1 A2;... Am; SEP B1 B2;...;Bn SEP
- CLS is a special token marking the class i.e., $\hat{y}$ as probability that sentence B follows A
- As shown below, there are 3 types of embedding: (1) token embedding; (2) segment embedding (sentence A or B); (3) positional embedding. These embedding are added elementwise.

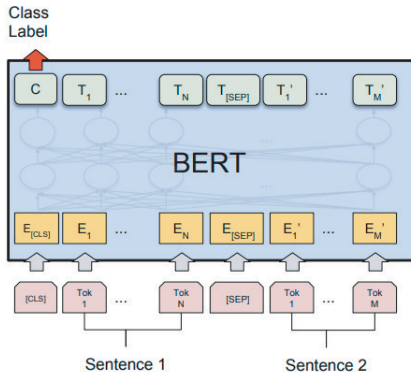


Fine-tuning supervised tasks 1 & 2

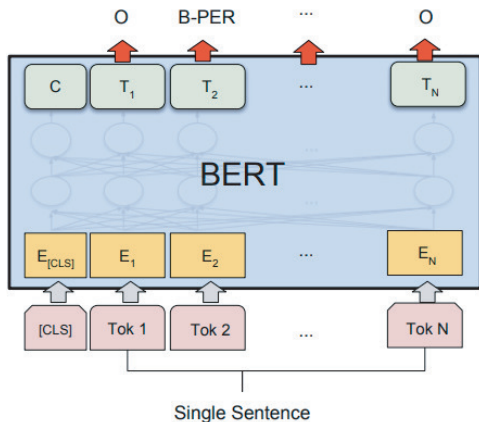
(1) Single sentence classification
(e.g., sentiment analysis)



(2) Sentence-pair classification (e.g.,
premise-hypothesis entailment)



Fine-tuning supervised task 3: Named Entity Recognition



- Shallow parsing. BIO notation: B is the beginning of an entity, I-x is for inside, and O is for outside any entity.
- It can be further extended to **Named Entity Recognition** to distinguish person, location, organization: {B-Per, I-Per, B-Loc, I-Loc, B-Org, I-Org, Outside}.
- This is useful for de-identification of personal information to protect their privacy while sharing useful information to the public.

For example,

Mrs	Green	spoke	today	in	New	York.	Green	chairs	the	finance	committee.
BP	IP	O	O	O	BL	IL	BP	O	O	O	O

Probing evaluates embedding layers as the inputs to the classifier

What is the best contextualized embedding for “**Help**” in that context?

For named-entity recognition task CoNLL-2003 NER

12 

...

7 

6 

5 

4 

3 

2 

1 



Help

First Layer

Embedding 

Dev F1 Score

91.0

Last Hidden Layer

12 

94.9

Sum All 12
Layers

12 
+
...
+
2 
+
1 
=

95.5

Second-to-Last
Hidden Layer

11 

95.6

Sum Last Four
Hidden

12 
+
11 
+
10 
+
9 
=

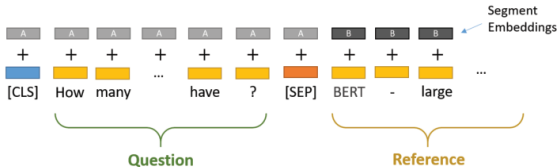
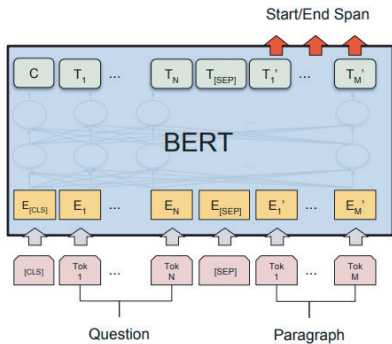
95.9

Concat Last
Four Hidden

9 10 11 12


96.1

Fine-tuning task 4: finding answer-containing paragraph with **prompts**



Question: How many parameters does BERT-large have?

Reference Text: BERT-large is really big... it has 24 layers and an embedding size of 1,024, for a total of 340M parameters! Altogether it is 1.34GB, so expect it to take a couple minutes to download to your Colab instance.

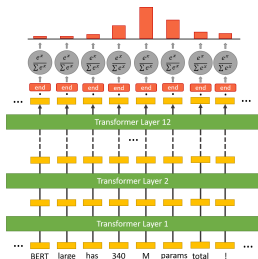
T: In meteorology, precipitation is any product of the condensation of atmospheric water vapor that falls under **gravity**. The main forms of precipitation include drizzle, rain, sleet, snow, **graupel** and hail... Precipitation forms as smaller droplets coalesce via collision with other rain drops or ice crystals **within a cloud**. Short, intense periods of rain in scattered locations are called "showers".

Q1: What causes precipitation to fall? A1: **gravity**

Q2: What is another main form of precipitation besides drizzle, rain, snow, sleet and hail? A2: **graupel**

Q3: Where do water droplets collide with ice crystals to form precipitation? A3: **within a cloud**

Fine-tuning task 4: finding answer-containing paragraph details



- Feed every token's final embedding into the start/end-token MLP classifier, which only has a single set of weights applied to every word.
- Take the start/end from MLP classifiers that have the highest scores among all words.
- The answer is the substring of the paragraph indexed by start and end.
- We can easily install BERT directly via `pip install transformers`

```
1 from transformers import BertForQuestionAnswering
2 model = BertForQuestionAnswering.from_pretrained('bert-large-...') # pretrained+fine-tuned
3 tokenizer = BertTokenizer.from_pretrained('bert-large-...')
4 question = 'What is Machine Learning?'
5 paragraph = 'ML is the scientific study of algorithms and statistical models ...'
6 encoding = tokenizer.encode_plus(text=question, text_pair=paragraph)
7 inputs = encoding['input_ids'] #Token embeddings
8 sentence_embedding = encoding['token_type_ids'] #Segment embeddings
9 start_score, end_score =
10     ↪ model(input_ids=torch.tensor([inputs]), token_type_ids=torch.tensor([sentence_embedding]))
11 start_index = torch.argmax(start_scores)
12 end_index = torch.argmax(end_scores)
13 tokens = tokenizer.convert_ids_to_tokens(inputs) #input tokens
14 answer = ' '.join(tokens[start_idx:end_idx+1])
```

[source](#)

BERT implementation and pre-trained BERT

- BERT models are usually large with millions of parameters. It is therefore quite expensive to pre-train them from scratch:
 - ▶ The BERT-base model uses 12 layers of transformer blocks with a hidden size of 768 and number of self-attention heads as 12 and has around 110M trainable parameters.
 - ▶ On the other hand, BERT-large uses 24 layers of transformer blocks with a hidden size of 1024 and number of self-attention heads as 16 and has around 340M trainable parameters!
- Because of that, Google have released their pre-trained BERT models hosted on [Colab](#), which can be fine-tuned for various tasks.
- The Google BERT code is also [open sourced](#).
- [PyTorch implementation of BERT](#) from hugging face is also open sourced.

Outline

Objectives

Sequential data modeling tasks

Recurrent neural network

Autoregressive neural network

Seq2Vec

Seq2Seq

Aligned case

Unaligned case

Backprop through time (or sequence)

LSTM

Attention mechanism

Transformers

Self-attention

Multi-headed attention

Positional encoding

Transformers Encoder

Comparing ConvNets, RNNs,
Self-attention

Pre-training deep language models

GPT

BERT

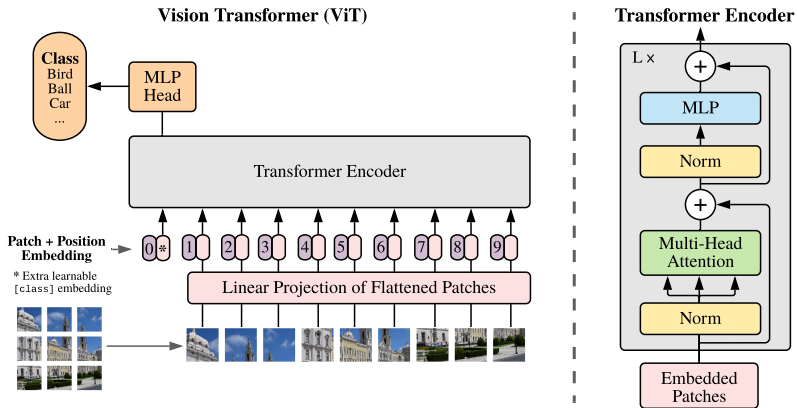
Beyond NLP

Computer Vision

Genomics

Summary

Vision Transformer [Dosovitskiy et al., 2020]

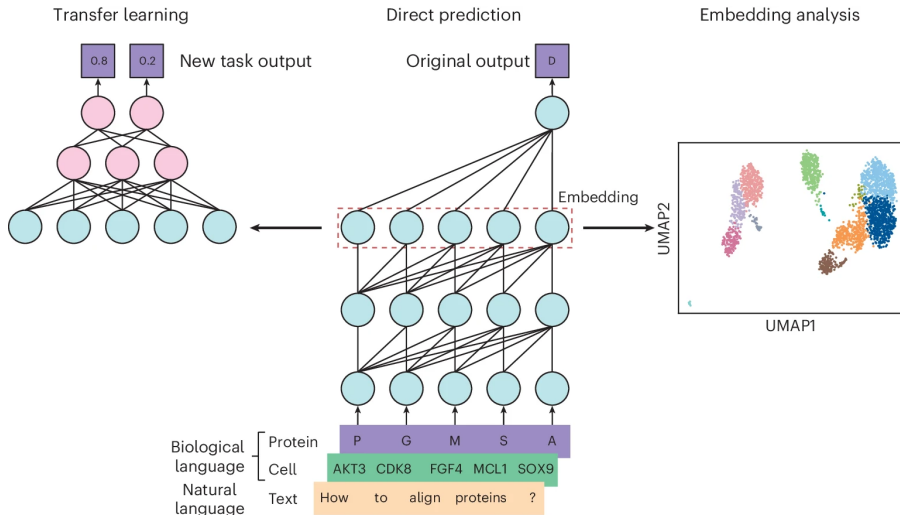


$$\mathbf{z}_0 = [\mathbf{x}_{class}; \mathbf{x}_p^1 \mathbf{E}; \mathbf{x}_p^2 \mathbf{E}; \dots; \mathbf{x}_p^N \mathbf{E}] + \mathbf{E}_{pos}, \quad \mathbf{E} \in \mathbb{R}^{(P^2 \cdot C) \times D}, \mathbf{E}_{pos} \in \mathbb{R}^{(N+1) \times D}$$

$$\mathbf{z}'_\ell = MHA(Norm(\mathbf{z}_{\ell-1})) + \mathbf{z}_{\ell-1}, \quad \mathbf{z}_\ell = MLP(Norm(\mathbf{z}'_\ell)) + \mathbf{z}'_\ell, \quad \ell = 1, \dots, L$$

$$\mathbf{y} = MLP(\mathbf{z}_{L,0})$$

LLM applications in genomics



Elana Simon, Kyle Swanson, and James Zou. Language models for biological research: a primer. *Nature Methods*, 21(8):1422–1429, 2024.

Summary

- **Sequential Data Modeling**

- ▶ Seq2Vec, Seq2Seq
- ▶ Classification, generation, translation

- **Recurrent Neural Networks**

- ▶ Autoregressive modeling
- ▶ Backpropagation through time
- ▶ GRU and LSTM to handle long dependencies

- **Attention Mechanisms**

- ▶ Dynamic context in decoding
- ▶ Query-Key-Value formulation
- ▶ Improves alignment in Seq2Seq

- **Transformer Architecture**

- ▶ Self-attention
- ▶ multi-head attention
- ▶ Positional encoding
- ▶ Highly parallel and scalable

- **Pre-trained Language Models**

- ▶ **GPT**: Transformer decoder
- ▶ **BERT**: Transformer encoder
- ▶ Task-specific fine-tuning

- **Applications Beyond NLP**

- ▶ **Vision Transformer (ViT)** for image understanding
- ▶ Genome language models